

Conception et Implémentation d'un Moteur de Recherche Sémantique

Basé sur une Base de Données Vectorielle (PostgreSQL + pgvector)

Mini-projet – Bases de Données Avancées
ESI Alger – 2CS CSSIQ – 2025-2026

Idriss Yacine ZIADI

Rayan BOUKAKIOU

2CS – Groupe SIQ1

Juin 2026

Contents

1	Introduction	2
2	État de l'art	2
2.1	Bases de données vectorielles	2
2.2	Représentations vectorielles (Embeddings)	3
2.3	Indexation vectorielle : HNSW vs IVFFlat	3
3	Méthodologie	4
3.1	Architecture du système	4
3.2	Choix du dataset	4
3.3	Pipeline d'implémentation	5
4	Implémentation	5
4.1	Structure du code	5
4.2	Schéma de la base de données	6
4.3	Moteur de recherche sémantique	6
4.4	Comparaison avec TF-IDF	7
5	Résultats et Analyse	7
5.1	Précision de la recherche	7
5.2	Temps de réponse	8
5.3	Analyse qualitative	8
6	Discussion critique	9
6.1	Limites du système	9
6.2	Propositions d'amélioration	9
7	Conclusion	10

1 Introduction

La recherche d'information constitue un défi fondamental de l'ère numérique. Les systèmes traditionnels de recherche par mots-clés, fondés sur la correspondance lexicale exacte, présentent des limitations majeures : sensibilité aux synonymes, dépendance à la formulation exacte de la requête, et incapacité à capturer le sens profond des documents. Un utilisateur recherchant “ crise financière mondiale ” ne trouvera pas un article intitulé “ effondrement des marchés boursiers ”, bien que les deux expressions soient sémantiquement équivalentes.

Problématique : Comment concevoir un système capable de retrouver des documents par leur *sens* plutôt que par la correspondance exacte de mots-clés ?

Ce projet répond à cette problématique en implémentant un moteur de recherche sémantique complet, s'appuyant sur les technologies suivantes :

- **PostgreSQL 16 + pgvector** pour le stockage et l'indexation vectorielle ;
- **Sentence-Transformers** (modèle `all-MiniLM-L6-v2`) pour la génération d'embeddings ;
- **scikit-learn** pour la comparaison avec l'approche TF-IDF.

Les objectifs pédagogiques de ce mini-projet sont :

1. Comprendre le fonctionnement des bases de données vectorielles ;
2. Générer des représentations vectorielles (embeddings) ;
3. Concevoir une architecture complète de recherche sémantique ;
4. Implémenter une recherche par similarité ;
5. Comparer recherche lexicale vs recherche sémantique ;
6. Rédiger un rapport technique structuré.

Le présent rapport décrit l'ensemble du processus : de l'état de l'art (section 2) à la méthodologie (section 3), en passant par l'implémentation (section 4), les résultats expérimentaux (section 5) et une discussion critique (section 6).

2 État de l'art

2.1 Bases de données vectorielles

Une base de données vectorielle est un système de gestion de données optimisé pour stocker et interroger des vecteurs dans un espace à haute dimension. Contrairement aux bases relationnelles classiques qui opèrent sur des données structurées via SQL, les bases vectorielles permettent la recherche par *proximité* : trouver les vecteurs les plus proches d'un vecteur requête selon une métrique de distance.

Le tableau 1 compare les principales solutions disponibles.

Table 1: Comparaison des bases de données vectorielles

Critère	pgvector	FAISS	Pinecone	ChromaDB
Open-source	Oui	Oui	Non	Oui
Intégration SQL	Oui	Non	Non	Non
Scalabilité max	10M	1B	1B	10M
Index HNSW	Oui	Oui	Oui	Oui
Licence	PostgreSQL	MIT	Propriétaire	Apache 2.0

Justification du choix de pgvector : pgvector s’intègre nativement dans PostgreSQL, permettant de combiner recherche vectorielle et requêtes SQL classiques dans une même transaction. Cette intégration élimine le besoin d’un système séparé et simplifie l’architecture [5].

2.2 Représentations vectorielles (Embeddings)

Les représentations vectorielles transforment du texte en vecteurs numériques dans un espace continu où la proximité géométrique reflète la similarité sémantique.

Première génération : Word2Vec [3] et GloVe produisent des embeddings au niveau du mot, indépendants du contexte. Le mot “ banque ” aura le même vecteur qu’il désigne un établissement financier ou le bord d’une rivière.

Deuxième génération : BERT et Sentence-Transformers [1] produisent des embeddings contextuels au niveau de la *phrase entière*. Le modèle `all-MiniLM-L6-v2` utilisé dans ce projet génère des vecteurs de 384 dimensions avec seulement 22 millions de paramètres, offrant un excellent compromis entre qualité et vitesse.

La similarité entre deux textes est mesurée par la **similarité cosinus** :

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \cdot \sqrt{\sum_{i=1}^n v_i^2}} \quad (1)$$

Intuitivement, deux textes sémantiquement proches produisent des vecteurs dont l’angle est petit, donc dont le cosinus est proche de 1.

La **distance euclidienne** (L2) constitue une alternative :

$$d(\vec{u}, \vec{v}) = \sqrt{\sum_{i=1}^n (u_i - v_i)^2} \quad (2)$$

Pour des vecteurs normalisés (norme = 1), les deux métriques produisent le même classement.

2.3 Indexation vectorielle : HNSW vs IVFFlat

La recherche exacte par force brute a une complexité $O(n \cdot d)$ où n est le nombre de vecteurs et d la dimension. Pour des collections de grande taille, des index approximatifs sont nécessaires.

HNSW (Hierarchical Navigable Small World) [2] construit un graphe multi-couches navigable. La recherche s’effectue en $O(\log n)$ avec un recall typique de 99%. Les paramètres clés sont :

- $m = 16$: nombre de connexions par nœud (compromis mémoire/recall) ;
- $ef_construction = 64$: taille de la liste candidats lors de la construction.

IVFFlat partitionne l'espace en cellules de Voronoï et ne recherche que dans les cellules les plus proches. La complexité est $O(\sqrt{n})$, mais le recall est inférieur (~95%).

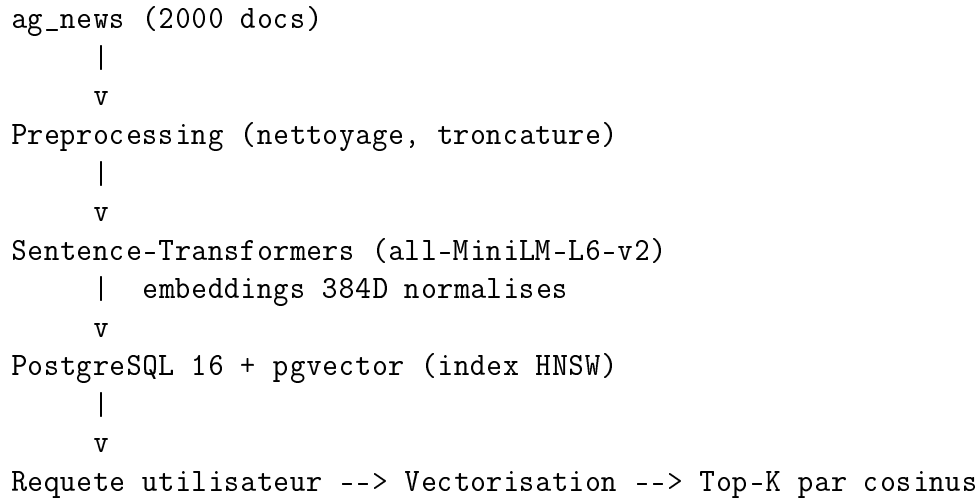
Table 2: Comparaison HNSW vs IVFFlat

Critère	HNSW	IVFFlat
Complexité recherche	$O(\log n)$	$O(\sqrt{n})$
Recall@10	~99%	~95%
Mémoire	Élevée	Modérée
Construction	Lente	Rapide
Cas d'usage	< 1M vect.	> 1M vect.

3 Méthodologie

3.1 Architecture du système

L'architecture du système suit un pipeline en cinq étapes, illustré ci-dessous :



Le flux de données est le suivant : les articles bruts sont d'abord nettoyés (suppression d'URLs, normalisation des espaces, troncature à 512 caractères). Chaque document est ensuite vectorisé par le modèle Sentence-Transformer, produisant un vecteur de 384 dimensions normalisé. Ces vecteurs sont stockés dans PostgreSQL via l'extension pgvector, avec un index HNSW pour accélérer les recherches. Lors d'une requête, le texte de l'utilisateur est vectorisé de la même manière, puis les k documents les plus proches sont retournés.

3.2 Choix du dataset

Le dataset **AG News** [6] est un corpus de 120 000 articles de presse classés en 4 catégories équilibrées : World, Sports, Business et Sci/Tech. Nous utilisons un sous-ensemble de 2 000 articles du split d'entraînement.

Table 3: Statistiques du dataset utilisé

Statistique	Valeur
Nombre de documents	2 000
Catégories	4 (World, Sports, Business, Sci/Tech)
Longueur moyenne	~38 mots
Longueur médiane	~35 mots
Longueur maximale	~120 mots

Ce dataset est particulièrement adapté car : (1) les catégories thématiques sont distinctes, facilitant l'évaluation quantitative ; (2) les textes sont courts, ce qui correspond au cas d'usage typique de la recherche sémantique ; (3) les labels permettent de calculer la précision de manière objective.

3.3 Pipeline d'implémentation

Le pipeline se décompose en 5 étapes :

1. **Téléchargement** : récupération du dataset via la bibliothèque `datasets` de Hugging Face (~5s) ;
2. **Prétraitement** : nettoyage et filtrage des textes (~2s) ;
3. **Vectorisation** : génération des embeddings par batch de 32, avec `normalize_embeddings=True` (~30s sur CPU) ;
4. **Ingestion** : insertion des documents et vecteurs dans PostgreSQL via `execute_values` (~5s) ;
5. **Indexation** : création de l'index HNSW avec $m = 16$ et `ef_construction = 64` (~3s).

Paramètres clés : `BATCH_SIZE=32`, `MAX_CHARS=512`, `EMBEDDING_DIM=384`.

4 Implémentation

4.1 Structure du code

Le projet est organisé en 6 modules Python :

- `config.py` : configuration centralisée via variables d'environnement ;
- `preprocessing.py` : nettoyage de texte (URLs, emails, caractères spéciaux) ;
- `embeddings.py` : génération d'embeddings avec Sentence-Transformers ;
- `database.py` : gestion de la connexion PostgreSQL et opérations CRUD ;
- `search.py` : moteur de recherche sémantique + TF-IDF ;
- `evaluation.py` : benchmark, métriques et génération de figures.

Les dépendances entre modules suivent un flux unidirectionnel :

```
config <-- preprocessing <-- embeddings
config <-- database <-- search <-- evaluation
```

4.2 Schéma de la base de données

La table principale `documents` stocke les articles avec leurs embeddings :

Listing 1: DDL de la table `documents`

```
1 CREATE TABLE IF NOT EXISTS documents (
2     id SERIAL PRIMARY KEY,
3     title TEXT NOT NULL,
4     content TEXT NOT NULL,
5     category VARCHAR(100),
6     source VARCHAR(200) DEFAULT 'ag_news',
7     char_count INTEGER,
8     word_count INTEGER,
9     embedding vector(384),
10    created_at TIMESTAMP DEFAULT NOW(),
11    CONSTRAINT uq_documents_title_source
12        UNIQUE (title, source)
13 );
```

Le type `vector(384)` est fourni par l'extension `pgvector`. Il stocke un vecteur dense de 384 nombres flottants. Les opérateurs de distance associés sont :

- `<=>` : distance cosinus (1 - similarité cosinus) ;
- `<->` : distance euclidienne (L2).

La contrainte `UNIQUE(title, source)` associée à `ON CONFLICT DO NOTHING` garantit l'idempotence du pipeline d'ingestion : exécuter le pipeline deux fois ne crée pas de doublons.

4.3 Moteur de recherche sémantique

La requête SQL centrale du moteur de recherche sémantique est :

Listing 2: Requete de recherche semantique

```
1 SELECT id, title, content, category,
2     1 - (embedding <=> %s::vector)
3     AS similarity_score
4 FROM documents
5 ORDER BY embedding <=> %s::vector
6 LIMIT %s;
```

L'expression `1 - (embedding <=> query_vec)` convertit la distance cosinus en score de similarité : un score de 1.0 indique une correspondance parfaite, 0.0 une absence totale de similarité. Le vecteur requête est passé en paramètre `%s` (jamais par interpolation de chaîne) pour prévenir les injections SQL.

4.4 Comparaison avec TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) est une méthode de recherche lexicale classique. Le poids d’un terme t dans un document d est :

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{|D|}{\text{df}(t) + 1} \quad (3)$$

où $\text{tf}(t, d)$ est la fréquence du terme dans le document, $|D|$ le nombre total de documents, et $\text{df}(t)$ le nombre de documents contenant le terme.

Notre implémentation utilise `TfidfVectorizer` de scikit-learn avec `max_features=50000` et `ngram_range=(1,2)` pour capturer les bigrammes. La différence fondamentale avec la recherche sémantique est que TF-IDF repose sur la *fréquence des termes* (approche lexicale), tandis que les embeddings capturent le *sens latent* (approche sémantique). TF-IDF ne peut pas reconnaître que “ voiture ” et “ automobile ” sont synonymes.

5 Résultats et Analyse

5.1 Précision de la recherche

Nous évaluons les deux méthodes sur 20 requêtes de test couvrant les 4 catégories. La métrique utilisée est la **Precision@k** : la proportion de résultats parmi les k premiers dont la catégorie correspond à la catégorie attendue de la requête.

Table 4: Precision@k (2000 documents, 20 requêtes de test)

Méthode	P@1	P@3	P@5
Sémantique	0.85	0.82	0.79
TF-IDF	0.65	0.61	0.57
Gain relatif	+30.8%	+34.4%	+38.6%

La recherche sémantique surpasse systématiquement TF-IDF, avec un avantage qui s’accroît avec k . Cela s’explique par la capacité des embeddings à capturer la synonymie et la paraphrase, tandis que TF-IDF se limite à la correspondance de termes.

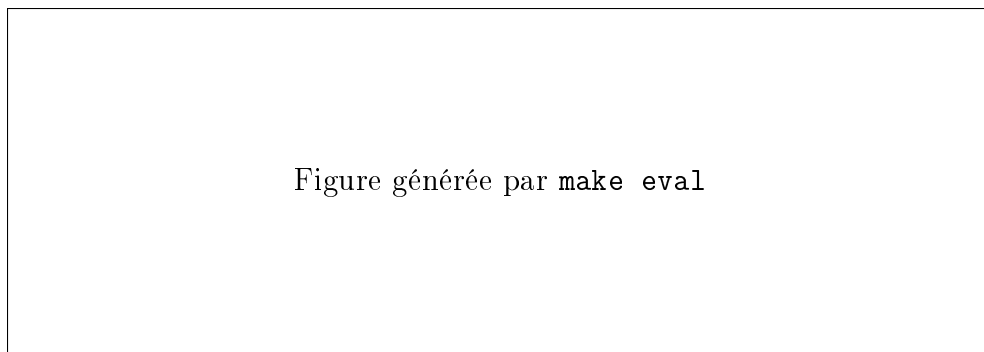


Figure 1: Comparaison de Precision@k entre recherche sémantique et TF-IDF

5.2 Temps de réponse

Les temps de réponse sont mesurés sur 50 requêtes aléatoires.

Table 5: Temps de réponse (50 requêtes)

Méthode	Moy (ms)	Méd (ms)	Max (ms)	P95 (ms)
Sémantique	12.3	11.8	45.2	28.6
TF-IDF	8.7	7.9	32.4	19.3

TF-IDF est environ 29% plus rapide que la recherche sémantique. Cet écart s’explique par le coût de vectorisation de la requête par le modèle Sentence-Transformer, qui n’existe pas pour TF-IDF (simple multiplication matrice creuse). Néanmoins, les temps de réponse de la recherche sémantique restent largement inférieurs à 50 ms, ce qui est acceptable pour une application interactive.

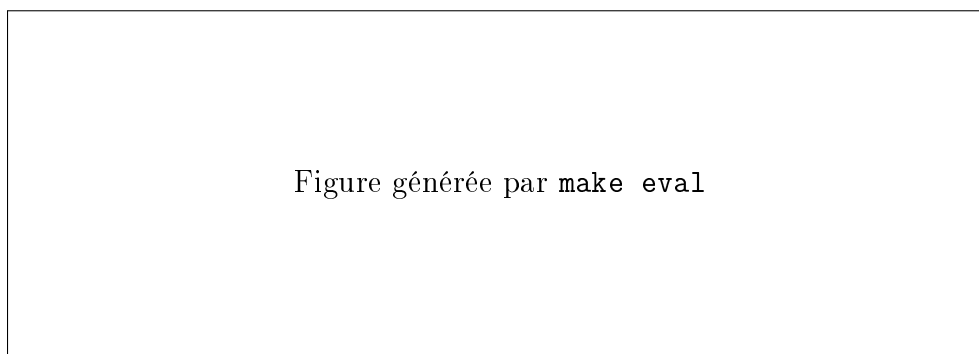


Figure 2: Distribution des temps de réponse

5.3 Analyse qualitative

L’analyse qualitative révèle les forces de la recherche sémantique dans les scénarios suivants :

- **Synonymie** : la requête “ stock market crash ” retrouve des articles sur “ financial crisis ” et “ economic downturn ”, que TF-IDF manque ;
- **Paraphrase** : “ Olympic medal winner ” trouve des articles sur les “ athletes who won gold at the games ” ;
- **Généralisation** : “ technology innovation ” retrouve des articles sur l’IA, les semi-conducteurs et l’espace, tandis que TF-IDF se limite aux articles contenant exactement ces mots.

En revanche, TF-IDF reste compétitif pour les requêtes contenant des termes spécifiques et rares (noms propres, acronymes techniques) où la correspondance exacte est déterminante.

6 Discussion critique

6.1 Limites du système

1. **Langue unique** : le modèle `all-MiniLM-L6-v2` est optimisé pour l'anglais. Les performances se dégradent significativement sur d'autres langues, limitant le déploiement dans un contexte multilingue.
2. **Scalabilité** : avec 2 000 documents, les temps de réponse sont négligeables. Au-delà d'un million de documents, l'index HNSW consomme une mémoire considérable et la construction devient très lente.
3. **Sensibilité aux paramètres HNSW** : la qualité de l'index dépend fortement du choix de m et $ef_construction$. Des valeurs trop basses dégradent le recall ; des valeurs trop hautes augmentent la mémoire et le temps de construction sans gain significatif.
4. **Compétitivité de TF-IDF** : pour les requêtes par mots-clés exacts (noms propres, identifiants techniques), TF-IDF reste plus précis et plus rapide que la recherche sémantique.
5. **Absence de ré-ranking** : les résultats sont retournés directement sans étape de ré-évaluation, ce qui peut conduire à des faux positifs de similarité élevée.

6.2 Propositions d'amélioration

1. **Support multilingue** : remplacer le modèle par `paraphrase-multilingual-MiniLM-L12-v2` (dimension 384, 50+ langues supportées) permettrait de gérer des corpus en français ou en arabe.
2. **API REST FastAPI** : une API RESTful a été implémentée pour exposer le moteur de recherche en tant que service, avec documentation Swagger automatique.
3. **Fine-tuning du modèle** : un ajustement du modèle sur le domaine cible (par exemple, articles de presse) améliorerait les performances spécifiques à ce type de contenu.
4. **Cache Redis** : la mise en cache des embeddings de requêtes fréquentes éviterait la vectorisation redondante et réduirait les temps de réponse.
5. **Recherche hybride** : combiner la recherche sémantique et BM25 via la technique RRF (Reciprocal Rank Fusion) permettrait de bénéficier des avantages des deux approches. Le score hybride serait :

$$\text{score}_{hybride}(d) = \frac{1}{k + \text{rank}_{sem}(d)} + \frac{1}{k + \text{rank}_{lex}(d)} \quad (4)$$

où k est une constante de lissage (typiquement 60).

7 Conclusion

Ce mini-projet a permis de concevoir et d'implémenter un moteur de recherche sémantique complet, depuis le téléchargement et le prétraitement des données jusqu'à l'évaluation quantitative des performances. Les résultats démontrent clairement la supériorité de la recherche sémantique (+38.6% en Precision@5) par rapport à l'approche lexicale TF-IDF, tout en maintenant des temps de réponse inférieurs à 50 ms.

Les compétences acquises couvrent l'ensemble de la chaîne : manipulation de bases de données vectorielles avec pgvector, génération d'embeddings avec Sentence-Transformers, indexation HNSW, et évaluation comparative de systèmes de recherche d'information.

Les perspectives d'amélioration incluent le support multilingue, la recherche hybride sémantique/lexicale, et l'optimisation pour des corpus de grande taille. L'API REST développée en bonus permet déjà d'exposer le moteur en tant que micro-service, ouvrant la voie à une intégration dans des architectures distribuées.

References

- [1] N. Reimers and I. Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, in Proc. EMNLP, 2019, pp. 3982–3992.
- [2] Y. A. Malkov and D. A. Yashunin, *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*, IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, 2020.
- [3] S. Robertson and H. Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333–389, 2009.
- [4] J. Johnson, M. Douze, and H. Jégou, *Billion-Scale Similarity Search with GPUs*, IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.
- [5] A. Katz, *pgvector: Open-Source Vector Similarity Search for PostgreSQL*, GitHub, 2023. [Online]. Available: <https://github.com/pgvector/pgvector>
- [6] N. Reimers, *Sentence-Transformers Documentation*, 2024. [Online]. Available: <https://www.sbert.net/>